



COMPLETE PROJECT DOCUMENTATION · JUNE 2026

Veto

Documentation

Architecture, Files, Logic, Deployment

The single source of truth for the Veto project — a deterministic, verifiable policy gate for AI agents that hold and move money on Sui. Covers the threat model (T1-T6), three layers of defense, Owner/Agent trust boundary, OwnerCap pattern, full file-by-file logic, API reference, database schema, Move module, testing, deployment, and the 20 hard judge questions with implementation answers.

23	T1-T6	19+10	20
SECTIONS	THREATS MITIGATED	TESTS PASSING	JUDGE Q&A

PROJECT	Veto — Policy Gate for AI Agents on Sui
BUILT FOR	Sui Overflow 2026 · Agentic Web Track
NETWORK	Sui Testnet (Move module = production target)
VERSION	v4 — with threat model, OwnerCap, test suite, Postgres fix

Table of Contents

1. What is Veto	3
2. The Pitch in One Paragraph	3
3. Evidence: Real AI Agents Currently Hold Wallets	3
4. The Threat Model (T1-T6)	4
5. The Three Layers of Defense	4
The confirmation flow (hallucination guard)	4
6. The Owner / Agent Trust Boundary	5
7. Why Sui Specifically (OwnerCap Pattern)	5
Four Sui-specific primitives	5
8. System Architecture	6
Request flow (the core pipeline)	7
9. Technology Stack	7
10. File Structure	8
11. File-by-File Logic	9
11.1 src/lib/policy-engine.ts — The Core	9
11.2 src/lib/vault.ts — On-chain Vault Simulator + Commit Logic	9
11.3 src/lib/sui.ts — Sui Integration	10
11.4 src/lib/llm.ts — Intent Parser	10
11.5 src/lib/auth.ts — Owner Authentication	11
11.6 src/lib/aliases.ts — Named Address Book	11
11.7 src/lib/types.ts — Shared Types	11
11.8 src/lib/db.ts — Prisma Client Singleton	11
11.9 src/app/page.tsx — Single-Page Dashboard	11
11.10 move/veto/sources/vault.move — The On-Chain Vault (Production Target)	12
12. API Reference	12
Example: Full Two-Step Flow	13
13. Database Schema	13
Rule table	13
AgentRequest table	13

RuleBookCommit table	14
14. The Move Module (Production Target)	14
OwnerCap pattern (the "why Sui" argument)	14
Atomic spend (race-condition safe)	14
Build & deploy (requires Sui CLI)	15
15. Environment Variables	15
16. Local Development	16
First-time setup	16
Available scripts	16
17. Testing	16
Unit tests (tests/policy-engine.test.ts)	16
API smoke test (tests/api-test.sh)	17
Manual test checklist (tests/manual-test-checklist.md)	17
18. Deployment	17
Before deploying to Vercel: switch to Postgres	17
Vercel deployment	17
Fund the agent wallet	17
19. Demo Script (5 minutes)	18
Demo scenarios (verified working)	18
20. Answers to 20 Hard Judge Questions	18
21. Who Buys This — Three Concrete Buyers	19
Monetization (stated plainly, not oversold)	20
22. Roadmap	20
23. Failure Modes & Operational Notes	20
Neon (Postgres) goes down	20
Sui testnet RPC issues	20
Faucet rate-limited	20
Owner forgets password	20
Move module deployment fails	21

1. What is Veto

Veto is a **deterministic, verifiable policy gate** that sits between an AI agent's reasoning and its on-chain wallet. Every proposed transaction must pass a fixed, human-defined rule book — evaluated in plain TypeScript code, not by another model — before it can be signed and submitted to the Sui blockchain. The rule book itself is fingerprinted on-chain, so even the rules can't be silently changed without it being visible.

Two roles exist even though one app runs both: the **Owner** (sets policy via /rules, authenticated) and the **Agent** (proposes actions via chat, untrusted). The deterministic policy engine is the wall between them. That wall, and its on-chain fingerprint, is the entire pitch.

2. The Pitch in One Paragraph

AI agents are starting to hold real wallets. Most agent frameworks let the model decide *and* execute in the same step. One bad instruction, one prompt injection, one hallucination, and funds move. **Veto puts two enforcement layers between an agent's reasoning and its wallet:** an off-chain deterministic policy engine (runtime, fast, editable) AND an on-chain vault (backstop, hard-capped, tamper-evident). Both must agree for a transaction to land. If the off-chain engine is compromised, the on-chain caps still hold. If a rule is silently edited, the on-chain commit hash diverges from what the feed shows was enforced.

The single sentence that matters: The off-chain policy engine is the runtime. The on-chain vault is the backstop. Both must agree for a transaction to land. If the off-chain engine is compromised, the on-chain caps still hold.

3. Evidence: Real AI Agents Currently Hold Wallets

This is not future-tense. It is a documented present-tense market. The "AI agents are starting to hold real wallets" claim is a documented fact, not a bet on the future.

Agent	What it is	Wallet capability
Truth Terminal	Claude-based autonomous AI agent (a16z-backed)	Holds GOAT token, autonomously promotes/posts, \$280K+ market cap
ElizaOS (ai16z Eliza)	Open-source agent framework	Native wallet plugins on Solana, Sui, Base; Stanford partnership
Coinbase Agentic Wallets	Launched Feb 11, 2026	MPC-secured wallet with programmable spending limits, session caps
Dynix, Cobo, Turnkey, Safe	Wallet infrastructure providers	All shipping agent-wallet products in 2025-2026

4. The Threat Model (T1-T6)

Every threat is named, mitigated, and (where possible) demo-able live. The two soft spots historically — T4 (rule book tampering) and T6 (Owner/Agent boundary) — are now the strongest demo moments in the project.

Threat	Scenario	Mitigation	Demo-able?
T1 Prompt injection	Agent reads untrusted external content with hidden instruction	Policy engine evaluates final intent regardless of how it was produced	Yes
T2 LLM hallucination	Model fabricates amount/recipient	zod validation + hard caps + two-step confirmation	Yes
T3 Compromised LLM response	MITM injects fake completion	Policy engine doesn't trust upstream — zero LLM imports	Architecturally shown
T4 Rule book tampering	Someone edits rules directly in DB	On-chain commit hash; UI recomputes local hash, shows red banner on mismatch	YES — verified live
T5 Replay / double-submit	Network retry executes same transfer twice	Idempotency key (hash of msg+amount+recipient), 60s window	Yes
T6 Owner/Agent boundary	Anyone could call /api/rules	OWNER_PASSWORD + signed cookie + on-chain OwnerCap (production)	YES — verified via curl

5. The Three Layers of Defense

Layer	What it does	Threat mitigated
1. Two-step confirmation (UI)	LLM parses intent -> user must explicitly confirm before any policy check or chain call	T2, T3
2. Off-chain policy engine (TS)	Deterministic rule checks. Zero API calls inside the policy function. Fail-closed when zero rules enabled.	T1
3. On-chain vault (Move)	Hard per_tx_cap + daily_cap enforced atomically in vault::spend(). OwnerCap pattern: protocol-level authorization.	T4, T6
4. Idempotency key (T5)	Hash of (message + amount + recipient) checked against recent EXECUTED requests. 60-second window.	T5

The confirmation flow (hallucination guard)

```
User types "send 100 sui to alice"
|
LLM parses -> {action: transfer, amountSui: 100, recipient: "alice"}
|
UI displays confirmation dialog:
"You said: send 100 sui to alice"
"Agent will execute: transfer 100 SUI -> 0x...0bad"
[diff warning if amount/recipient differs from what was mentioned]
|
User clicks "Confirm & execute" or "Reject"
|
Only then does the policy engine + on-chain vault check + SUI execution run
```

6. The Owner / Agent Trust Boundary

Two roles, one app, in v1: the **Owner** edits the rule book via `/rules` (every change re-commits the rule hash on-chain, authenticated via `x-owner-token` header in v1, NextAuth + zkLogin in v1.1). The **Agent** is the chat/LLM path — it can **ONLY** propose actions via `/api/agent/message`, with no route, permission, or code path that touches `/api/rules`.

The deterministic policy engine sits between them. **The Agent literally cannot modify the rules** — the `requireOwner()` middleware in `src/lib/auth.ts` rejects any request to `/api/rules*` without the owner token. Verified live in the network panel during demo.

In production (Move deployed), the boundary is enforced one layer deeper at the chain itself: the `commit_rules()` and `configure()` functions take `_: &OwnerCap` as their first argument. The Sui runtime rejects any tx that doesn't include the `OwnerCap` object **BEFORE** the function body runs.

7. Why Sui Specifically (OwnerCap Pattern)

The honest version of "why Sui": nothing in the basic pitch couldn't run on any chain with a `require(msg.sender == owner)` check. The Sui-specific version is the **OwnerCap capability pattern**.

The Move module is designed so updating the rule registry requires *possessing a capability object*, not just passing a permission check in code:

```
public struct OwnerCap has key, store {}

public fun commit_rules(
  _cap: &OwnerCap,
  registry: &mut Vault,
  new_hash: vector<u8>,
) {
  registry.rules_commit_hash = new_hash;
  registry.rules_version = registry.rules_version + 1;
}
```

On an account-based chain (Ethereum, Solana, etc.), "only the owner can do this" lives entirely inside mutable application code. On Sui, possessing the right object *is* the authorization — the runtime checks object ownership before your Move code even runs. A transaction that doesn't include the `OwnerCap` literally cannot call `commit_rules` or `configure`, full stop, at the protocol level.

This is demo-able as fact, not asserted as a slide: try the call without the cap in a Sui CLI terminal, show it get rejected on-chain, clip it or do it live. The app-level password (`OWNER_PASSWORD` env var + cookie) is for convenience — the actual authority boundary is enforced by the chain itself.

Four Sui-specific primitives

- **Shared objects** — `vault::spend()` is a single atomic Move transaction protected by consensus (race-condition prevention)
- **Move resource safety** — funds inside the vault literally cannot be moved except via the vault's entry function (impossible in Solidity's storage model)
- **Sponsored transactions** — for v1.1 user-delegated wallets
- **OwnerCap capability pattern** — protocol-level authorization, not app-level

8. System Architecture

```
+-----+ message +-----+
| Chat UI | -----> | POST /api/agent | <- Agent role
| (Agent) | | /message | (no owner cookie)
+-----+ +-----+
| 1. LLM parse (zod-validated)
v
status = AWAITING_CONFIRMATION
|
v
+-----+
| User confirms | <- hallucination guard (T2)
| parsed intent | (2-step flow)
+-----+
| POST /api/agent/confirm
v
1a. IDEMPOTENCY CHECK (T5)
hash(msg+amount+recipient)
reject if EXECUTED in last 60s
|
v
2. ON-CHAIN VAULT pre-flight
(per_tx_cap, daily_cap)
|
v
3. OFF-CHAIN policy engine
(allowlist, denylist) - zero LLM calls (T1, T3)
|
+-----+
v fail v pass
BLOCKED 4. Sign + execute via
(no chain call) @mysten/sui (real testnet tx)
|
v
Persist + UI live feed

+-----+ login +-----+
| /rules UI | -----> | POST /api/owner | <- Owner role
| (Owner) | | /login | (OWNER_PASSWORD
+-----+ | -> session cookie | -> signed cookie)
| +-----+
| edit rule (cookie)
v
+-----+
| POST/PATCH | <- requireOwner() middleware
| /api/rules | validates cookie OR x-owner-token
+-----+
| 5. Recompute SHA-256(rules JSON)
v
+-----+
| commit_rules(OwnerCap, ...) | <- In production: Sui runtime
| on Vault object | rejects tx if OwnerCap object
| (simulated in v1) | is not included (T6 enforced
+-----+ at the protocol level)
|
v
+-----+
```

```
| T4: tamper detection | <- On every GET /api/rules:
| recompute hash, compare | recompute local hash,
| to last commit | compare to last committed,
| -> tampered: boolean | show red banner if mismatch
+-----+
```

Request flow (the core pipeline)

1. **User types** a plain-English instruction (send 1 sui to alice)
2. **POST /api/agent/message** — LLM parses into {action, amountSui, recipient}, zod-validates the shape, resolves aliases to real Sui addresses, stores the request as AWAITING_CONFIRMATION, returns the parsed intent + diff to the UI
3. **Confirmation dialog** shows original message + parsed intent side-by-side. If amount differs from any number mentioned, amber diff warning. User clicks "Confirm & execute" or "Reject"
4. **POST /api/agent/confirm** runs the full pipeline: T5 idempotency check, on-chain vault pre-flight, off-chain policy engine (fail-closed if zero rules), SUI execution if all pass
5. **Activity feed** updates live (polls every 4s), showing every request with status badge, parsed intent, failing rule or tx digest + explorer link

9. Technology Stack

Layer	Choice	Why
Framework	Next.js 16 (App Router) + TypeScript	Modern, Vercel-native, serverless-friendly
UI	Tailwind CSS + shadcn/ui	Polished components, consistent design, fast to build
DB (local dev)	Prisma + SQLite	Zero-config offline dev
DB (production)	Prisma + PostgreSQL on Neon (free tier)	SQLite does NOT work on Vercel — serverless filesystem resets per request
Chain SDK	@mysten/sui v2 (SuiJsonRpcClient, Ed25519Keypair, Transaction)	Current Sui SDK (not deprecated @mysten/sui.js)
On-chain	Move module veto::vault	Production target; off-chain simulator mirrors semantics
Wallet model	App-custodied single testnet keypair, server-side only	v1 simplification — v1.1 adds user-delegated wallets via dapp-kit
LLM	z-ai-web-dev-sdk (swappable)	Already integrated in the env; abstracted so any provider works
Validation	zod on every LLM output and API input	Defense in depth — never trust unvalidated data
Auth	Owner password + signed HMAC session cookie (v1) -> on-chain OwnerCap (production)	Cookie for UX, OwnerCap for actual security boundary
Idempotency	SHA-256 of (message + amount + recipient), 60s window	Prevents replay/double-submit (T5)
Hosting	Vercel (Hobby/free) + Neon (free Postgres)	Both have free tiers, no credit card required for demo scale

10. File Structure

```
veto/
|-- README.md <- Project-level overview
|-- VETO_COMPLETE_DOCUMENTATION.md <- This file
|-- package.json <- Scripts: dev, build, test, test:api, pre-deploy
|-- vitest.config.ts <- Test config with @/ path alias
|-- next.config.ts <- Next.js 16 config
|-- tsconfig.json <- TypeScript config
|-- tailwind.config.ts <- Tailwind theme
|-- eslint.config.mjs <- ESLint rules
|-- .env / .env.example <- Environment variables (gitignored / template)
|-- .gitignore
|
|-- prisma/
| |-- schema.prisma <- ACTIVE schema (sqlite or postgres)
| |-- schema.sqlite.prisma <- Local dev only - DO NOT DEPLOY
| `-- schema.postgres.prisma <- Production target (Vercel-safe)
|
|-- move/ <- Move smart contract source
| `-- veto/
| | |-- Move.toml <- Package config
| | `-- sources/
| `-- vault.move <- The on-chain vault (OwnerCap pattern)
|
|-- scripts/ <- Operational scripts
| |-- switch-db.sh <- Swap prisma schema between sqlite/postgres
| `-- pre-deploy-check.sh <- Fails loudly if SQLite is active
|
|-- tests/ <- Test suite
| |-- policy-engine.test.ts <- 19 unit tests for the rule logic (vitest)
| |-- api-test.sh <- 10-step curl smoke test for the live API
| `-- manual-test-checklist.md <- Browser + chain-state verification checklist
|
|-- src/
|-- app/ <- Next.js App Router
| |-- layout.tsx <- Root layout (fonts, Toaster)
| |-- page.tsx <- Single-page dashboard (3 tabs + dialogs)
| |-- globals.css <- Tailwind + custom styles
| `-- api/ <- API routes
| |-- agent/
| | |-- message/route.ts <- Step 1: LLM parse -> AWAITING_CONFIRMATION
| | `-- confirm/route.ts <- Step 2: idempotency + vault + policy + execute
| |-- owner/
| | |-- login/route.ts <- Password -> signed session cookie
| | |-- logout/route.ts <- Clear cookie
| | `-- status/route.ts <- { authenticated: boolean }
| |-- requests/route.ts <- Activity feed (GET)
| |-- rules/
| | |-- route.ts <- List + create (owner-only on POST)
| | `-- [id]/route.ts <- Toggle/edit/delete (owner-only)
| |-- wallet/route.ts <- Agent wallet address + balance (GET)
| |-- aliases/route.ts <- Named address book (GET)
| `-- seed/route.ts <- Seed default rules + initial commit
|
|-- lib/ <- Business logic
| |-- policy-engine.ts <- THE CORE: pure TS, zero LLM calls
| |-- vault.ts <- On-chain vault simulator + commit + tamper detection
```

```
| |-- sui.ts <- Sui testnet client + keypair + transfer
| |-- llm.ts <- Intent parser (z-ai-web-dev-sdk + zod)
| |-- auth.ts <- Owner password + signed HMAC cookie
| |-- aliases.ts <- Named address book
| |-- types.ts <- Shared TypeScript types
| `-- db.ts <- Prisma client singleton
|
|-- components/ui/ <- shadcn/ui components
```

11. File-by-File Logic

11.1 src/lib/policy-engine.ts — The Core

This is the heart of the project. **Pure, synchronous, side-effect-free TypeScript.** No LLM call happens inside this module — that sentence is the whole pitch.

Types:

```
type ParsedIntent = { action: "transfer"; amountSui: number; recipient: string };
type PolicyContext = { spentTodaySui: number };
type PolicyDecision =
| { decision: "APPROVED" }
| { decision: "BLOCKED"; failedRule: string; reason: string };

// 4 rule types:
// - MAX_AMOUNT_PER_TX -> { maxAmountSui: number }
// - DAILY_SPEND_CAP -> { capSui: number }
// - DENYLIST_ADDRESS -> { addresses: string[] }
// - ALLOWED_RECIPIENT -> { addresses: string[] }
```

Key design decisions:

- **Zero LLM imports** — the file imports nothing from llm.ts. Architecturally enforced: the policy engine is the last line of defense and cannot be influenced by model output.
- **Fail-closed on empty rule book** — an empty/misconfigured rule book must NOT mean "allow everything." If you want to allow everything, add an explicit MAX_AMOUNT_PER_TX rule with a very high cap.
- **Deterministic order** — rules are sorted by createdAt, so the audit trail is reproducible.
- **First-failure-wins** — only the first failing rule is reported, with a single clear reason. No cascading error noise.

11.2 src/lib/vault.ts — On-chain Vault Simulator + Commit Logic

Mirrors the semantics of move/veto/sources/vault.move exactly. In v1 (current): runs off-chain. In production: the same off-chain code calls vault::spend() instead of the local simulator, and the on-chain enforcement becomes authoritative.

Key functions:

```
// Default vault config: 5 SUI per-tx cap, 20 SUI daily cap
const DEFAULT_VAULT_CONFIG = {
  perTxCapMist: 5n * 1_000_000_000n,
  dailyCapMist: 20n * 1_000_000_000n,
};
```

```

// Get current vault state (computed from DB)
async function getVaultState(): Promise<VaultState>

// T4: tamper detection - recompute local hash, compare to last commit
async function detectTampering(): Promise<{ tampered, currentHash, committedHash,
... }>

// Compute SHA-256 hash of the canonical rule set (sorted, enabled-only)
function computeRulesHash(rules: Rule[]): string

// Commit the current rule set (simulated: stores hash + version in DB)
// Returns commitDurationMs so UI can show "committed in X.Xs"
async function commitRulesToVault(rules): Promise<VaultCommit & { commitDurationMs
}>

// Pre-flight check: would this spend be allowed by the on-chain vault?
async function preflightVaultSpend(amountMist: bigint): Promise<VaultSpendResult>

```

11.3 src/lib/sui.ts — Sui Integration

Server-side only. The agent's testnet keypair is loaded from env and never sent to the client.

```

import { SuiJsonRpcClient, getJsonRpcFullnodeUrl } from "@mysten/sui/jsonRpc";
import { Ed25519Keypair } from "@mysten/sui/keypairs/ed25519";
import { Transaction } from "@mysten/sui/transactions";
import { MIST_PER_SUI } from "@mysten/sui/utils";

// Singleton client (lazy-initialized)
function getSuiClient(): SuiJsonRpcClient

// Singleton keypair (loaded from SUI_AGENT_SECRET_KEY env var)
function getAgentKeypair(): Ed25519Keypair

// Agent wallet address (derived from keypair)
function getAgentAddress(): string

// Get balance in whole SUI (not MIST)
async function getAgentBalanceSui(): Promise<number>

// THE ONLY FUNCTION THAT SIGNS ANYTHING.
// Only called AFTER policy engine approves.
async function executeTransfer(recipient, amountSui): Promise<TransferResult>

```

Key implementation detail: `executeTransfer` uses `tx.splitCoins(tx.gas, [amount]) + tx.transferObjects([coin], recipient)`. The agent's own gas coin is the source, so no separate coin management needed. Pre-flight balance check prevents obvious failures.

11.4 src/lib/llm.ts — Intent Parser

One job: turn free-text user input into a structured `ParsedIntent`, or flag it as unparseable. The model's output is treated as untrusted — it goes through zod validation before being used anywhere downstream.

Key design decisions:

- **Never throws** — on any error (LLM call fails, JSON parse fails, zod validation fails), returns `{action: "unknown", reason: "..."}.` The caller surfaces this to the user.

- **Markdown fence stripping** — LLMs sometimes wrap JSON in ```json fences despite instructions. Handle it gracefully.
- **JSON extraction** — if the model adds commentary around the JSON, extract just the {...} part.
- **zod validation** — schema check is the last gate before the intent is trusted.

11.5 src/lib/auth.ts — Owner Authentication

Two layers of authorization: app-level (v1) OWNER_PASSWORD + signed HMAC session cookie; chain-level (production) OwnerCap object on Sui.

Key security details:

- **HMAC-signed cookie** — the cookie value is .. Tampering with expiresAt invalidates the HMAC.
- **Constant-time comparison** — `diff |= sig.charCodeAt(i) ^ expected.charCodeAt(i)` prevents timing attacks.
- **HttpOnly + SameSite=Strict** — cookie can't be read by JavaScript, can't be sent on cross-site requests.
- **Backwards compat** — x-owner-token header still works for API clients (curl tests, CI).

11.6 src/lib/aliases.ts — Named Address Book

Lets the demo say "send 5 SUI to alice" instead of pasting hex addresses on camera. Edit ALIASES with your own testnet addresses. `resolveAlias(input)` returns the Sui address for a known alias, or the input itself if it's already a valid 0x-prefixed address, or null if unresolvable.

11.7 src/lib/types.ts — Shared Types

RuleType union, Rule interface (config is unknown — handles both Postgres Json object and SQLite JSON string), RequestStatus union, plus RULE_TYPE_LABELS and RULE_TYPE_DESCRIPTIONS for UI display.

11.8 src/lib/db.ts — Prisma Client Singleton

Prevents multiple Prisma client instances in dev (Next.js hot reloading would otherwise create one per reload). Uses `globalThis` to cache the client across hot reloads.

11.9 src/app/page.tsx — Single-Page Dashboard

The main UI. Three tabs + two dialogs:

Dashboard tab:

- Wallet card (address, balance, TESTNET badge, explorer link)
- On-chain vault card (per-tx cap, daily cap, spent today, current commit hash + version, SIMULATED badge)
- Chat input (Agent role, plain English -> SUI transfer)
- Activity feed (live-updating, polls every 4s, color-coded status badges)

Rule book tab:

- T4 tamper detection banner (red, fires on hash mismatch)
- Owner authentication banner (green when authenticated, amber when not)
- On-chain rule book commit card (version, full SHA-256 hash, caps, spent today)

- Off-chain rule book (list with toggle/delete buttons, "Add rule" dialog)

Architecture tab:

- Updated ASCII diagram showing the full flow (idempotency, vault, policy, OwnerCap, tamper detection)
- Stack list (framework, UI, DB, chain, on-chain, LLM, auth, idempotency, tamper detection)
- 20-question Q&A; section (every hard judge question with a specific implementation answer)

Dialogs:

- Confirmation dialog (two-step flow): original message + parsed intent side-by-side, amber diff warning, Reject / Confirm & execute buttons
- Owner login dialog: password input, demo password hint, OwnerCap pattern explanation

11.10 move/veto/sources/vault.move — The On-Chain Vault (Production Target)

Key properties:

- **OwnerCap pattern** — possession of the OwnerCap object IS authorization. Sui runtime checks before function runs.
- **Atomic spend (race-condition safe)** — vault::spend() checks daily cap AND increments spent counter in the same Move transaction. Sui's shared-object consensus serializes concurrent calls.
- **Events for off-chain audit** — Spent, RulesCommitted, CapsConfigured events emitted on every state change.
- **Read-only views** — per_tx_cap_mist(), daily_cap_mist(), spent_today_mist(), rules_commit_hash(), rules_version() callable by anyone.

12. API Reference

Method	Path	Auth	Purpose
POST	/api/agent/message	None (Agent)	LLM parse -> stage as AWAITING_CONFIRMATION
POST	/api/agent/confirm	None (Agent)	Idempotency (T5) -> vault -> policy -> SUI execution
GET	/api/requests?limit=20	None	Activity feed (newest first)
GET	/api/rules	None (read)	List rules + vault state + commit + tamper flag
POST	/api/rules	Owner cookie/token	Create rule -> triggers vault re-commit
PATCH	/api/rules/:id	Owner cookie/token	Toggle/edit -> triggers vault re-commit
DELETE	/api/rules/:id	Owner cookie/token	Delete -> triggers vault re-commit
POST	/api/owner/login	None	Verify password -> set signed session cookie
POST	/api/owner/logout	None	Clear session cookie
GET	/api/owner/status	None	Returns { authenticated: boolean }
GET	/api/wallet	None	Read-only wallet info (address, balance, network)

Method	Path	Auth	Purpose
GET	/api/aliases	None	Known recipient aliases
POST	/api/seed	None	Seed default rules + initial commit (idempotent)

Example: Full Two-Step Flow

```
# Step 1: Send message, get parsed intent + request ID
curl -X POST http://localhost:3000/api/agent/message \
-H "Content-Type: application/json" \
-d '{"message":"send 1 sui to alice"}'
# Response: { "id":"cmq...", "parsedIntent":{...},
"status":"AWAITING_CONFIRMATION" }

# Step 2: Confirm (or reject)
curl -X POST http://localhost:3000/api/agent/confirm \
-H "Content-Type: application/json" \
-d '{"id":"cmq...", "decision":"confirm"}'
# Response: { "id":"cmq...", "status":"EXECUTED", "txDigest":"..." }
# or: { "id":"cmq...", "status":"BLOCKED", "failedRule":"..." }
```

13. Database Schema

Rule table

Column	Type	Description
id	String (cuid)	Primary key
name	String	Human-readable rule name
type	String	MAX_AMOUNT_PER_TX DAILY_SPEND_CAP ALLOWED_RECIPIENT DENYLIST_ADDRESS
config	Json (Postgres) / String (SQLite)	Rule-specific: {maxAmountSui} {capSui} {addresses:[...]}
enabled	Boolean	Whether the rule is active (default: true)
createdAt	DateTime	Creation timestamp
updatedAt	DateTime	Last update timestamp

AgentRequest table

Column	Type	Description
id	String (cuid)	Primary key
rawMessage	String	The original plain-English instruction
parsedIntent	Json?	The LLM-parsed intent (null if parsing failed)
amountSui	Float?	SUI amount (null if not a transfer)
recipient	String?	Resolved Sui address (null if not a transfer)

Column	Type	Description
status	String	PENDING AWAITING_CONFIRMATION APPROVED BLOCKED EXECUTED FAILED
failedRule	String?	Rule name if BLOCKED (e.g. Per-transaction cap, on_chain_vault:..., fail_closed_no_rules, idempotency_check, user_rejected)
failReason	String?	Human-readable failure reason
txDigest	String?	Sui transaction digest if EXECUTED
confirmedAt	DateTime?	When user confirmed (null until confirmation step)
createdAt	DateTime	Creation timestamp

RuleBookCommit table

Column	Type	Description
id	String (cuid)	Primary key
commitHash	String	SHA-256 of canonical rule set JSON (0x-prefixed hex)
version	Int	Monotonically increasing version number
txDigest	String?	null in simulator mode; real Sui tx digest once Move is deployed
createdAt	DateTime	Commit timestamp

14. The Move Module (Production Target)

The Move source at `move/veto/sources/vault.move` defines a vault that **actually holds funds** and enforces hard caps on-chain. Key properties:

OwnerCap pattern (the "why Sui" argument)

```
public struct OwnerCap has key, store {}

public fun commit_rules(_cap: &OwnerCap, vault: &mut Vault, new_hash: vector<u8>) {
    vault.rules_commit_hash = new_hash;
    vault.rules_version = vault.rules_version + 1;
    event::emit(RulesCommitted { hash: new_hash, version: vault.rules_version });
}
```

On Sui, the runtime checks object ownership **BEFORE** the function runs. A tx without the OwnerCap is rejected at the protocol level — not by app code that could be patched or bypassed.

Atomic spend (race-condition safe)

```
public fun spend(vault: &mut Vault, coin: Coin<SUI>, recipient: address,
    amount_mist: u64, ctx: &mut TxContext) {
    assert!(amount_mist > 0, EAmountZero);
    assert!(amount_mist <= vault.per_tx_cap_mist, EAmountExceedsPerTx);

    // Roll daily window if 24h elapsed
    let now_ms = tx_context::timestamp_ms(ctx);
    if (now_ms - vault.window_start_ms >= 24 * 60 * 60 * 1000) {
```

```

vault.spent_today_mist = 0;
vault.window_start_ms = now_ms;
};

// ATOMIC: check + increment in same transaction
let projected = vault.spent_today_mist + amount_mist;
assert!(projected <= vault.daily_cap_mist, EAmountExceedsDailyCap);
vault.spent_today_mist = projected;

// Split + transfer
let to_send = coin::split(&mut coin, amount_mist, ctx);
transfer::public_transfer(to_send, recipient);
transfer::public_transfer(coin, tx_context::sender(ctx));

event::emit(Spent { recipient, amount_mist });
}

```

Sui's shared-object consensus serializes concurrent calls to `spend()`. Two simultaneous spends CANNOT both pass — one will see the other's increment and reject.

Build & deploy (requires Sui CLI)

```

sui move build --path move/veto
sui client publish --gas-budget 100000000 move/veto

# From the publish output, set these env vars:
# VAULT_OBJECT_ID - the shared Vault object ID
# VAULT_PACKAGE_ID - the package ID
# OWNER_CAP_ID - the OwnerCap object ID (transferred to your deployer address)

```

15. Environment Variables

Variable	Required	Description	Example
DATABASE_URL	Yes	SQLite (file:) for local dev OR Postgres (postgresql://) for production	postgresql://user:pass@host/db?sslmode=require
SUI_AGENT_SECRET_KEY	Yes	Ed25519 private key for the agent's testnet wallet	suiprivkey1q...
SUI_NETWORK	Yes	Sui network to use	testnet
OWNER_PASSWORD	Yes	Password for POST /api/owner/login	dev-owner-password
OWNER_TOKEN	Optional	Bearer token for API clients - alternative to cookie	dev-owner-token
OWNER_COOKIE_SECRET	Optional	HMAC secret for signing session cookies	random-32-byte-hex
VAULT_OBJECT_ID	Production	Shared Vault object ID from sui client publish	0x...
VAULT_PACKAGE_ID	Production	Package ID from publish	0x...
OWNER_CAP_ID	Production	OwnerCap object ID - kept server-side only	0x...

16. Local Development

First-time setup

```
# 1. Install dependencies
bun install

# 2. Copy env template and edit values
cp .env.example .env
# Edit .env: set SUI_AGENT_SECRET_KEY, OWNER_PASSWORD

# 3. Switch to SQLite schema for local dev
./scripts/switch-db.sh sqlite

# 4. Push schema to local SQLite DB
bun run db:push

# 5. Start dev server
bun run dev
```

Open <http://localhost:3000> — the app auto-seeds three default rules + initial vault commit on first load.

Available scripts

Command	What it does
bun run dev	Start Next.js dev server on port 3000
bun run lint	Run ESLint
bun run test	Run unit tests (vitest) - 19 tests
bun run test:watch	Run unit tests in watch mode
bun run test:api	Run API smoke test (requires dev server running)
bun run pre-deploy	Run pre-deploy safety check (catches SQLite + missing env vars)
bun run db:push	Push Prisma schema to DB
bun run db:switch-postgres	Switch to Postgres schema + push
bun run db:switch-sqlite	Switch to SQLite schema + push
bun run build	Production build

17. Testing

Unit tests (tests/policy-engine.test.ts)

19 tests covering all 4 rule types (boundary conditions, just-over, just-under), multi-rule behavior (first failure wins, all-pass approves, disabled rules ignored), and the fail-closed edge case (zero enabled rules -> BLOCKED with fail_closed_no_rules).

```
bun run test
# Expected: 19 passed, 0 failed
```

API smoke test (tests/api-test.sh)

10-step curl-based test verifying all 6 threat mitigations end-to-end: T4 (tamper detection), T5 (idempotency), T6 (auth), on-chain vault block path, and two-step confirmation flow.

```
# Against localhost
BASE_URL=http://localhost:3000 OWNER_PASSWORD=dev-owner-password bun run test:api
# Expected: 10 passed, 0 failed

# Against deployed Vercel URL
BASE_URL=https://your-app.vercel.app OWNER_PASSWORD=yourpassword bun run test:api
```

Manual test checklist (tests/manual-test-checklist.md)

Everything that needs a browser or real chain state: Auth (T6) login/logout flow, Tamper detection (T4) with both SQLite and Postgres mutation commands, OwnerCap enforcement on-chain, Core flow BLOCKED vs APPROVED vs real tx digest, Idempotency (T5) with funded wallet, Fail-closed edge case, Deployment verification. SQLite-on-Vercel warning is at the top.

18. Deployment

Before deploying to Vercel: switch to Postgres

SQLite does NOT work on Vercel. Serverless functions get a fresh filesystem on every request — a SQLite file written to disk doesn't persist. Your rule book and history will silently reset in production even though everything works on localhost.

```
# 1. Create a free Postgres instance at https://neon.tech
# 2. Set DATABASE_URL in .env to the pooled Neon connection string:
# postgresql://user:pass@host/db?sslmode=require

# 3. Switch the active schema to Postgres + push
./scripts/switch-db.sh postgres
bun run db:push

# 4. Run the pre-deploy check (catches SQLite + missing env vars)
./scripts/pre-deploy-check.sh
```

Vercel deployment

1. **Push to GitHub** (public repo) — .env and *.db are gitignored
2. **Import on Vercel** (Hobby/free tier, no credit card required)
3. **Set environment variables** in Vercel: DATABASE_URL (Postgres), SUI_AGENT_SECRET_KEY, OWNER_PASSWORD, SUI_NETWORK=testnet
4. **Deploy**
5. **Smoke test the live URL:** BASE_URL=https://your-app.vercel.app
OWNER_PASSWORD=yourpassword bun run test:api — should be 10/10 PASS

Fund the agent wallet

The agent's testnet wallet needs SUI for the EXECUTED flow to actually land on-chain. Get the agent address from the dashboard, visit <https://faucet.testnet.sui.io>, request testnet SUI to that address. If unfunded, the

EXECUTED flow still runs through the policy engine and on-chain vault pre-flight, then returns a meaningful "insufficient balance" error — proving the entire pipeline works. The BLOCKED flows are completely unaffected (they never touch the chain).

19. Demo Script (5 minutes)

Time	Content	Criterion targeted
0:00-0:30	Problem: Agents holding real money, most frameworks let the model decide and execute in one step	Real-World Application
0:30-1:00	Mechanism: deterministic check, not another model's opinion, between proposal and signature. Name the Owner/Agent split explicitly	Real-World Application
1:00-3:00	Live demo: (a) send 100 sui to alice -> BLOCKED by on-chain vault, no chain call; (b) send 0.5 sui to self -> EXECUTED, real Explorer link; (c) Edit a rule on /rules -> new on-chain commit hash appears	Product & UX + Technical
3:00-4:00	Tamper detection: open terminal, run sqlite3 UPDATE -> switch back to browser -> red banner fires within 15s	Technical Implementation
4:00-5:00	Why this is honest: real working version of an idea Sui flagged as missing. Roadmap: user-delegated wallets, multi-action, enterprise case	Presentation & Vision

Demo scenarios (verified working)

Scenario	Input	Expected outcome
Block by on-chain vault	send 100 sui to alice	BLOCKED - on-chain vault: EAmountExceedsPerTx (100 > 5 SUI per-tx cap)
Block by off-chain rule	send 1 sui to 0x...0bad	BLOCKED - blocked by: Known-bad address blocklist
Reject in confirmation	send 2 sui to self -> click Reject	BLOCKED - rejected by: user rejected
Fail-closed	Disable all rules -> send 0.1 sui to self	BLOCKED - fail_closed_no_rules
Successful execution	send 0.5 sui to self (funded wallet)	EXECUTED - real tx digest + Sui Explorer link

20. Answers to 20 Hard Judge Questions

The Architecture tab in the app contains a complete Q&A; section. Here's the summary — every answer maps to a specific implementation decision visible in the code.

#	Question	Answer (implementation)
1	Name 3 AI agents with wallets	Truth Terminal, ElizaOS, Coinbase Agentic Wallets — README evidence table
2	Why "deterministic" valuable?	Architectural separation: LLM proposes in /api/agent/message, code enforces in /api/agent/confirm — different modules, different auth
3	Which frameworks let LLM decide+execute?	LangGraph, ElizaOS, Goose — all execute on model decision. Coinbase AgentKit ships limits but inside their closed stack
4	Why blockchain vs Git commit?	Git proves what was committed, not what ran. On-chain commit + spend() tied to commit = provable runtime enforcement

#	Question	Answer (implementation)
5	Who is the attacker?	Three: the agent (prompt injection), compromised dependencies (defeated by on-chain vault), compromised backend operator (defeated by on-chain caps)
6	Owner sets max=1M -> drain?	On-chain vault caps regardless. Owner can't bypass without vault::configure() (visible event)
7	Why Move/Sui?	Shared objects (atomic spend), Move resource safety, sponsored tx, OwnerCap capability pattern (protocol-level auth)
8	Race condition?	vault::spend() is atomic — Sui consensus serializes concurrent calls
9	Hallucination prevention?	Two-step confirmation dialog with diff warnings. Zod validates schema; user validates semantics
10	Who enforces Owner/Agent?	requireOwner() middleware (cookie or token) + on-chain OwnerCap (production)
11	Vercel compromised?	On-chain vault is backstop. Compromised evaluateRules() cannot exceed caps
12	Who buys this?	Agent framework teams (ElizaOS, ai16z, LangChain) — they build agents, not policy
13	Market size?	~500-2000 deployment teams today, growing 3x/year (LangChain, Coinbase AgentKit metrics)
14	Competition?	AgentKit (closed), Permit.io (off-chain), Arcjet (rate limit), LangGraph HITL (doesn't scale). Veto: on-chain enforcement
15	App-custodied = centralized?	In v1 yes, but on-chain vault is the security boundary. v1.1 = user-delegated wallets
16	Remove chain, what disappears?	Tamper-evidence + race prevention + backend-compromise survival
17	OpenAI adds max_spend -> startup dies?	What survives: open, framework-agnostic, multi-chain, on-chain enforcement
18	Why install Veto vs if(amount > limit)?	Your code can be edited by anyone. The on-chain vault cannot.
19	Neon goes down?	App fails gracefully. No funds at risk (chain vault still enforces). Fail-closed design.
20	Why rank above team with zkLogin but no on-chain commit?	Their limits are claims, ours are proofs. For Sui specifically, "verifiable policy enforcement" is exactly what Sui said is missing

21. Who Buys This — Three Concrete Buyers

DAOs and treasury teams

Delegating limited authority to an agent for routine ops (recurring payments, rebalancing, yield farming). They need enforceable, provable limits before a governance vote will approve delegating anything real. The on-chain vault + tamper-evident rule commit is exactly the audit trail their governance committee asks for.

Agent framework providers (ElizaOS, ai16z, LangChain, CrewAI)

Distribution path: bundle Veto's policy layer as a default safety module inside an agent SDK, the way payment processors bundle fraud checks rather than making every merchant build their own. Coinbase already shipped this internally as part of AgentKit — Veto is the open, framework-agnostic version.

Custodians and regulated entities piloting agentic execution

Compliance teams need an auditable control layer, and "capability-enforced, on-chain-verifiable policy" is language they can actually evaluate — unlike "we trust the model." The enterprise wedge: regulated capital can't move via an AI agent without a provable policy layer between the agent and the wallet.

Monetization (stated plainly, not oversold)

Open-source the policy engine for adoption and trust. Charge for a hosted multi-agent dashboard and compliance export (CSV/PDF audit reports tied to on-chain commit hashes). Standard open-core, easy for a judge to believe.

22. Roadmap

v1.1 — Delegated user wallets via dapp-kit + sponsored transactions

Let users connect their own Sui wallet and delegate a spending-limited sub-key to the agent. The on-chain vault stays the same.

v1.2 — Multiple action types

Beyond transfers — Navi deposits, DeepBook trades, NFT mints — each with its own rule types and vault spend paths.

v2.0 — Multi-agent

Multiple agents with separate sub-vaults under one Owner, each with its own caps and rule book commits.

Long-term

Regulated entities (custodians, treasuries, DAOs) that need provable guardrails before they'll let any AI agent near real funds.

23. Failure Modes & Operational Notes

Neon (Postgres) goes down

App fails gracefully — no requests can be approved or blocked because the off-chain engine can't load rules. No funds at risk — the chain vault still enforces its cap (vault state is on-chain, not in Neon). Recovery: bring Neon back up, app resumes. Agent requests during the outage are queued by the user (no auto-retry). This is a **fail-closed** design, not fail-open.

Sui testnet RPC issues

Wallet balance reads may fail — UI shows "Loading wallet..." indefinitely. Transfer execution may fail — AgentRequest gets status=FAILED with the Sui error message. BLOCKED flows are unaffected (they never touch the chain).

Faucet rate-limited

Agent wallet stays at 0 SUI. EXECUTED flow returns "insufficient balance" error. All BLOCKED flows still work perfectly (they never touch the chain). Fix: fund the wallet from a clean IP, or use a different faucet.

Owner forgets password

Cannot edit rules until password is reset. Agent can still propose actions (and they'll be evaluated against the existing rule book). Fix: set a new OWNER_PASSWORD env var, restart the server.

Move module deployment fails

The off-chain simulator continues to work (v1 mode). The UI shows "SIMULATED" badge — judges know the on-chain deployment is pending. Fix: deploy the Move module when Sui CLI is available, set VAULT_OBJECT_ID / VAULT_PACKAGE_ID / OWNER_CAP_ID env vars.

Built for Sui Overflow 2026 — Agentic Web track. Sui's own current ecosystem messaging explicitly frames agent guardrails and verifiable policy enforcement as missing infrastructure. Veto is a focused, single-mechanism answer to exactly that problem — built on Sui's primitives (shared objects, Move resources, atomic spend, OwnerCap capability pattern) that no other chain replicates.